# Assignment 2

## BENCHMARKING

**B)** On a GPU, the forward pass takes 38.35ms, 75.02ms, and 115.84ms for small, medium, and large models respectively, with the backward pass taking approximately 2–2.5× longer (. Standard deviations are small (≤1ms) for medium and large, indicating low noise.

| Size | Forward (ms) | Forward+Backward (ms) | Backward only (ms) |
|---|---|---|---|
| small | 38.35 ± 2.71 | 86.62 ± 0.96 | ~48.27 |
| medium | 75.02 ± 0.61 | 182.92 ± 1.93 | ~107.90 |
| large | 115.84 ± 0.94 | 327.54 ± 0.89 | ~211 |

**C)** With no warmup, forward pass std is ~52ms, reflecting overhead in the first few steps. By 2 warmup steps, std drops to ~1ms and the mean stabilizes.

## NSYS_PROFILE

**A)** The total GPU time across all kernels is ~224ms across all steps (10 steps), so roughly ~22ms per forward pass. It is lower than the 37.36ms measured by Python's timeit, with the ~22ms gap accounted for by Python overhead.

**B)** ampere_sgemm_128x64_tn takes 50.5% of total GPU time, 555 invocations total. Divide by your step count (10 steps + 5 warmup = 15) that is ~37 invocations per forward pass. This is the matrix multiply kernel and it stays dominant in backward too.

**C)** elementwise_kernel (MulFunctor) 6.1%, elementwise_kernel (copy) 2.9%, vectorized_elementwise_kernel (add) 1.9%, reduce_kernel (MeanOps) 1.2%, sigmoid_kernel 0.7%, reduce_kernel (MaxOps) 1.0%

**D)** During a full training step, the fraction of GPU time spent on matrix multiplication drops from ~78% to ~54%. while optimizer-related elementwise kernels and gradient ops become significant, collectively accounting for ~22% of GPU time that doesn't exist in inference.

**E)** The softmax operation in the forward pass totals ~6.2ms, compared to ~174ms for matrix multiplications. The FLOP difference is much larger: matmuls perform $O(N^2 d)$ multiply-accumulate operations versus softmax's $O(N^2)$ additions and exponentials.

## MIXED PRECISION ACCUMULATION

Accumulating 1000 additions of 0.01 in FP32 gives the correct result (10.0001 ≈ 10.0). Accumulating in FP16 gives 9.9531 due to FP16's limited precision and rounding errors at small values. Keeping the accumulator in FP32 but adding FP16 values (cases 3 and 4) recovers accuracy (10.0021). Demonstrating why mixed-precision training keeps accumulations and reductions in FP32 while computing matrix multiplications in lower precision.

## BENCHMARKING MIXED PRECISION

A)

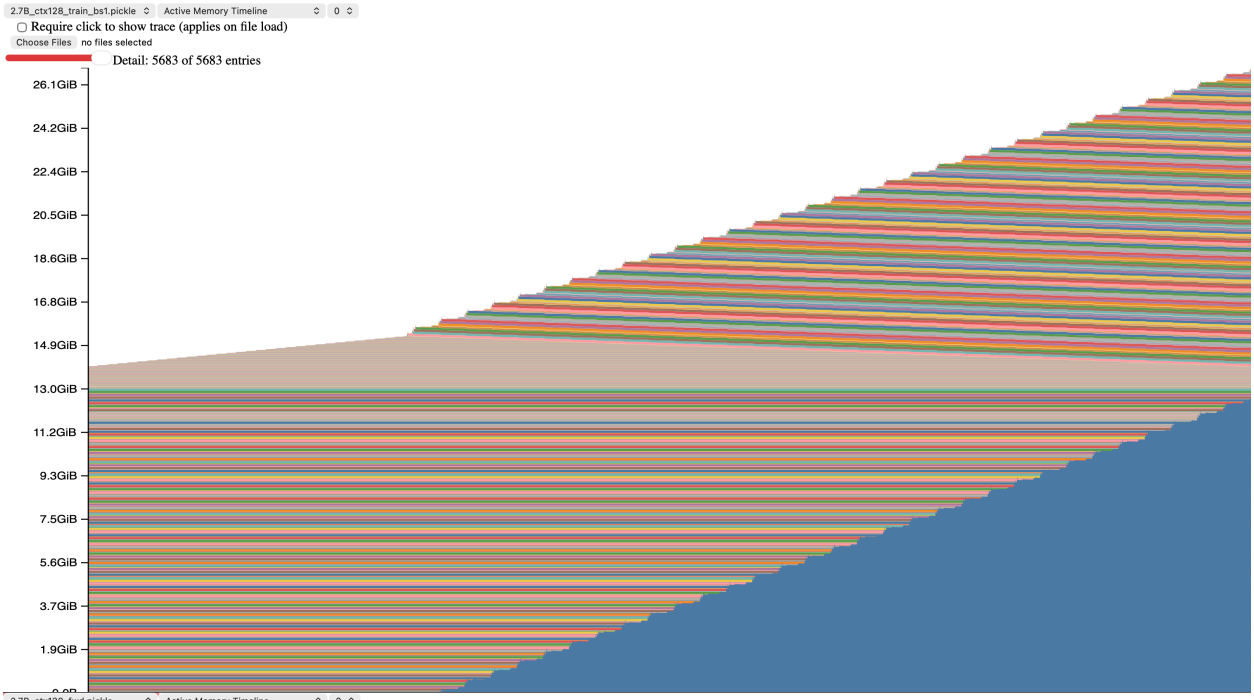
- **Model parameters:** FP32
- **Output of fc1:** FP16
- **Output of Layer Norm:** FP32
- **Predicted logits:** FP16
- **Loss:** FP32
- **Gradients**: FP32

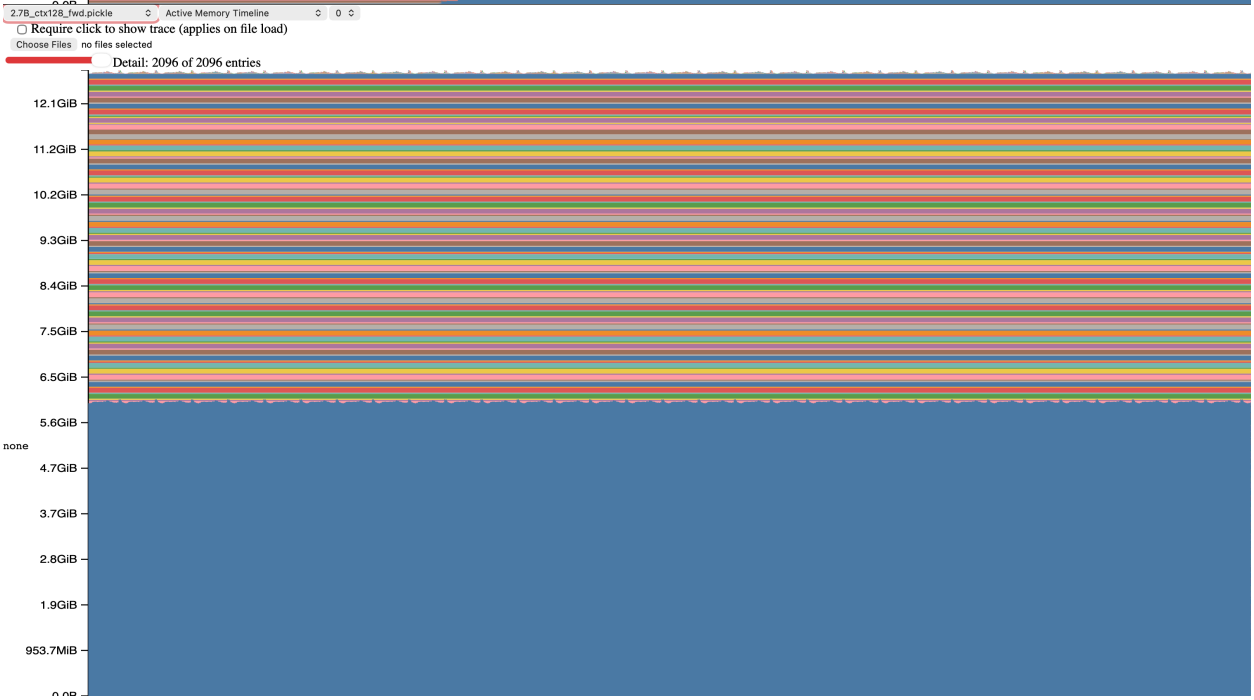
**B)** LayerNorm involves summing squared values which is sensitive to FP16's limited dynamic range and can produce inaccurate normalization statistics or overflow/underflow for large or small activation values. BF16 is less critical than FP16 because it has the same dynamic range as FP32 meaning it is far less prone to overflow or underflow even though it has lower precision than FP32.

**C)** small (83.17ms to 92.84ms), medium (174.40ms to 198.68ms), and large (322.03ms to 316.57ms) with BF16 actually being slower for small and medium models. This overhead likely stems from autocasting and precision conversion costs outweighing Tensor Core benefits when models are not sufficiently compute-bound. The trend suggests meaningful BF16 speedups would only emerge at larger scales (xl, 2.7B) where matrix multiplications dominate heavily.

## MEMORY_PROFILING

**A)** The forward-only timeline (Image 2, 2.7B_ctx128_fwd) shows a flat profile at ~12.9 GiB throughout. Reflecting that only model weights and a fixed set of per-layer activations are live. The training step timeline (2.7B_ctx128_train_bs1) shows a continuous upward ramp reaching ~26 GiB, with two identifiable phases. First half (blue region) corresponding to the forward pass where intermediate activations are retained in memory for use during backprop, and the second half (colorful bands) corresponds to the backward pass where gradient tensors accumulate layer by layer as gradients are computed in reverse order.



**B) note batch size  = 1**

| Context Length | Forward Pass (batch=4) | Training Step (batch=1)* |
|---|---|---|
| 128 | 13,232.8 MB | 27,614.2 MB |
| 256 | 13,351.0 MB | 29,185.7 MB |
| 512 | 13,823.6 MB | 33,162.5 MB |

**C)** BF16 mixed precision does not reduce peak memory usage in fact it increases it for the forward pass (FP32: 13,233 MB vs BF16: 19,609 MB at ctx=128), and the training step OOMs under BF16 even at batch_size=1 while FP32 succeeds at 27,614 MB. Meaningful memory reduction from mixed precision would require explicitly storing parameters and optimizer states in BF16, not just autocast alone.

**D)** At the reference hyperparameters (batch_size=4, context_length=128, d_model=2560), a single residual stream activation tensor in FP32 has shape (4, 128, 2560), giving $4 \times 128 \times 2560 \times 4 = 5{,}242{,}880$ bytes. Dividing by $1024^2 = 1{,}048{,}576$, this equals 5.0 MB.

**E)** The largest allocations shown at reduced detail are 100 MB each, corresponding to the feed-forward network weight matrices of shape d_model × d_ff = 2560 × 10240 in FP32.

## PYTORCH ATTENTION

**A)**

| d_model | seq_len | forward_ms | backward_ms |
|---|---|---|---|
| 16 | 256 | 0.39 | 1.25 |
| 16 | 1024 | 0.75 | 2.23 |
| 16 | 4096 | 7.61 | 26.04 |
| 16 | 8192 | 28.34 | 97.05 |
| 16 | **16384** | **OOM** | **OOM** |
| 32 | 256 | 0.39 | 1.25 |
| 32 | 1024 | 0.68 | 2.16 |
| 32 | 4096 | 7.91 | 26.52 |
| 32 | 8192 | 29.28 | 98.87 |
| 32 | **16384** | **OOM** | **OOM** |
| 64 | 256 | 0.39 | 1.26 |
| 64 | 1024 | 0.71 | 2.22 |
| 64 | 4096 | 8.36 | 27.43 |
| 64 | 8192 | 31.10 | 102.52 |
| 64 | **16384** | **OOM** | **OOM** |
| 128 | 256 | 0.39 | 1.27 |
| 128 | 1024 | 0.77 | 2.35 |
| 128 | 4096 | 9.23 | 29.23 |
| 128 | 8192 | 34.54 | 109.56 |
| 128 | **16384** | **OOM** | **OOM** |

OOM errors occur at seq_len=16384 for all head dimensions, as the attention score matrix of shape (batch=8, seq=16384, seq=16384) alone requires ~8 GiB, and saving it for the backward pass pushes total memory to ~49 GiB, exceeding the A100's 40 GiB. Memory before the backward pass scales quadratically with sequence length.

doubling seq from 4096 to 8192 increases memory ~4×, confirming the $O(seq^2)$ cost of storing the attention matrix P. To eliminate this memory cost, we can use FlashAttention, which recomputes attention scores tile-by-tile during the backward pass instead of saving the full seq×seq matrix to HBM.

## TORCH COMPILE

**A)**

| d_model | seq_len | fwd uncompiled (ms) | fwd compiled (ms) | bwd uncompiled (ms) | bwd compiled (ms) | status |
|---|---|---|---|---|---|---|
| 16 | 256 | 0.41 | 0.30 | 1.28 | 0.91 | ok |
| 16 | 1024 | 0.67 | 0.60 | 2.12 | 1.54 | ok |
| 16 | 4096 | 7.63 | 4.04 | 26.20 | 14.73 | ok |
| 16 | 8192 | 28.45 | 16.84 | 97.03 | 54.67 | ok |
| 16 | 16384 | OOM | 56.75 | OOM | 208.88 | compile ok! |
| 32 | 256 | 0.40 | 0.39 | 1.31 | 1.01 | ok |
| 32 | 1024 | 0.67 | 0.53 | 2.13 | 1.60 | ok |
| 32 | 4096 | 7.84 | 5.10 | 26.62 | 16.48 | ok |
| 32 | 8192 | 29.32 | 18.38 | 98.90 | 58.52 | ok |
| 32 | 16384 | OOM | 62.36 | OOM | 219.58 | compile ok! |
| 64 | 256 | 0.40 | 0.39 | 1.34 | 1.00 | ok |
| 64 | 1024 | 0.71 | 0.65 | 2.22 | 1.66 | ok |
| 64 | 4096 | 8.28 | 4.97 | 27.43 | 16.45 | ok |
| 64 | 8192 | 31.04 | 17.15 | 102.46 | 58.08 | ok |
| 64 | 16384 | OOM | OOM | OOM | OOM | OOM |
| 128 | 256 | 0.40 | 0.38 | 1.33 | 1.02 | ok |
| 128 | 1024 | 0.77 | 0.71 | 2.35 | 1.75 | ok |
| 128 | 4096 | 9.17 | 5.87 | 29.39 | 18.42 | ok |
| 128 | 8192 | 34.54 | 20.62 | 109.51 | 65.04 | ok |
| 128 | 16384 | OOM | OOM | OOM | OOM | OOM |

**B)**

| Size | Uncompiled (ms) | Compiled (ms) |
|---|---|---|
| small | 86.62 | 41.14 |
| medium | 182.92 | 121.41 |
| large | 327.54 | 255.83 |

## FLASH BENCHMARKING

**A) NOTE the results are for batch size 1.**

| seq_len | d | dtype | pytorch_fwd_ms | pytorch_bwd_ms | pytorch_total_ms | triton_fwd_ms | triton_bwd_ms | triton_total_ms |
|---|---|---|---|---|---|---|---|---|
| 128 | 16 | float32 | 0.133 | 0.513 | 0.646 | 0.013 | 0.514 | 0.527 |
| 128 | 16 | bfloat16 | 0.155 | 0.523 | 0.678 | 0.010 | 0.509 | 0.519 |
| 128 | 32 | float32 | 0.143 | 0.518 | 0.662 | 0.013 | 0.609 | 0.623 |
| 128 | 32 | bfloat16 | 0.140 | 0.521 | 0.662 | 0.012 | 0.609 | 0.622 |
| 128 | 64 | float32 | 0.148 | 0.513 | 0.662 | 0.015 | 0.595 | 0.610 |
| 128 | 64 | bfloat16 | 0.145 | 0.532 | 0.678 | 0.013 | 0.596 | 0.609 |
| 128 | 128 | float32 | 0.141 | 0.501 | 0.642 | 0.017 | 0.590 | 0.607 |
| 128 | 128 | bfloat16 | 0.150 | 0.532 | 0.682 | 0.016 | 0.608 | 0.625 |
| 256 | 16 | float32 | 0.145 | 0.510 | 0.655 | 0.014 | 0.634 | 0.648 |
| 256 | 16 | bfloat16 | 0.141 | 0.541 | 0.682 | 0.017 | 0.641 | 0.658 |
| 256 | 32 | float32 | 0.145 | 0.535 | 0.680 | 0.019 | 0.622 | 0.641 |
| 256 | 32 | bfloat16 | 0.144 | 0.558 | 0.702 | 0.017 | 0.641 | 0.658 |
| 256 | 64 | float32 | 0.144 | 0.535 | 0.679 | 0.023 | 0.636 | 0.658 |
| 256 | 64 | bfloat16 | 0.155 | 0.538 | 0.692 | 0.018 | 0.640 | 0.658 |
| 256 | 128 | float32 | 0.147 | 0.555 | 0.702 | 0.026 | 0.652 | 0.678 |
| 256 | 128 | bfloat16 | 0.157 | 0.538 | 0.694 | 0.025 | 0.623 | 0.647 |
| 512 | 16 | float32 | 0.138 | 0.563 | 0.701 | 0.023 | 0.646 | 0.669 |
| 512 | 16 | bfloat16 | 0.147 | 0.555 | 0.702 | 0.024 | 0.637 | 0.660 |
| 512 | 32 | float32 | 0.151 | 0.542 | 0.692 | 0.032 | 0.624 | 0.655 |
| 512 | 32 | bfloat16 | 0.151 | 0.556 | 0.707 | 0.027 | 0.618 | 0.645 |
| 512 | 64 | float32 | 0.153 | 0.551 | 0.703 | 0.038 | 0.628 | 0.666 |
| 512 | 64 | bfloat16 | 0.153 | 0.550 | 0.702 | 0.030 | 0.645 | 0.675 |
| 512 | 128 | float32 | 0.153 | 0.528 | 0.681 | 0.045 | 0.624 | 0.669 |
| 512 | 128 | bfloat16 | 0.142 | 0.541 | 0.683 | 0.042 | 0.601 | 0.643 |
| 1024 | 16 | float32 | 0.156 | 0.556 | 0.712 | 0.038 | 0.653 | 0.691 |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 1024 | 16 | bfloat16 | 0.157 | 0.543 | 0.699 | 0.039 | 0.623 | 0.662 |
| 1024 | 32 | float32 | 0.155 | 0.547 | 0.701 | 0.056 | 0.611 | 0.668 |
| 1024 | 32 | bfloat16 | 0.146 | 0.553 | 0.699 | 0.046 | 0.618 | 0.665 |
| 1024 | 64 | float32 | 0.156 | 0.539 | 0.694 | 0.069 | 0.614 | 0.683 |
| 1024 | 64 | bfloat16 | 0.144 | 0.528 | 0.673 | 0.053 | 0.590 | 0.643 |
| 1024 | 128 | float32 | 0.140 | 0.542 | 0.682 | 0.083 | 0.573 | 0.656 |
| 1024 | 128 | bfloat16 | 0.134 | 0.526 | 0.660 | 0.076 | 0.566 | 0.642 |
| 2048 | 16 | float32 | 0.196 | 0.478 | 0.674 | 0.086 | 0.580 | 0.666 |
| 2048 | 16 | bfloat16 | 0.146 | 0.541 | 0.687 | 0.082 | 0.571 | 0.653 |
| 2048 | 32 | float32 | 0.200 | 0.494 | 0.693 | 0.122 | 0.552 | 0.674 |
| 2048 | 32 | bfloat16 | 0.153 | 0.534 | 0.686 | 0.099 | 0.550 | 0.649 |
| 2048 | 64 | float32 | 0.230 | 0.470 | 0.700 | 0.157 | 0.523 | 0.680 |
| 2048 | 64 | bfloat16 | 0.150 | 0.527 | 0.677 | 0.116 | 0.536 | 0.651 |
| 2048 | 128 | float32 | 0.287 | 0.456 | 0.742 | 0.209 | 0.479 | 0.688 |
| 2048 | 128 | bfloat16 | 0.151 | 0.521 | 0.672 | 0.167 | 0.480 | 0.647 |
| 4096 | 16 | float32 | 0.642 | 0.841 | 1.483 | 0.216 | 0.663 | 0.879 |
| 4096 | 16 | bfloat16 | 0.410 | 0.454 | 0.863 | 0.197 | 0.458 | 0.654 |
| 4096 | 32 | float32 | 0.662 | 0.849 | 1.510 | 0.282 | 0.708 | 0.989 |
| 4096 | 32 | bfloat16 | 0.411 | 0.427 | 0.838 | 0.221 | 0.445 | 0.667 |
| 4096 | 64 | float32 | 0.768 | 1.087 | 1.855 | 0.370 | 1.000 | 1.370 |
| 4096 | 64 | bfloat16 | 0.420 | 0.438 | 0.858 | 0.261 | 0.415 | 0.676 |
| 4096 | 128 | float32 | 1.004 | 1.607 | 2.610 | 0.538 | 1.631 | 2.169 |
| 4096 | 128 | bfloat16 | 0.438 | 0.474 | 0.912 | 0.385 | 0.411 | 0.796 |
| 8192 | 16 | float32 | 2.333 | 3.246 | 5.579 | 0.648 | 2.428 | 3.076 |
| 8192 | 16 | bfloat16 | 1.397 | 1.593 | 2.990 | 0.562 | 1.278 | 1.840 |
| 8192 | 32 | float32 | 2.432 | 3.189 | 5.621 | 0.785 | 2.613 | 3.399 |
| 8192 | 32 | bfloat16 | 1.396 | 1.565 | 2.960 | 0.629 | 1.273 | 1.902 |
| 8192 | 64 | float32 | 2.866 | 4.289 | 7.155 | 1.069 | 3.878 | 4.947 |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 8192 | 64 | bfloat16 | 1.445 | 1.607 | 3.052 | 0.726 | 1.376 | 2.102 |
| 8192 | 128 | float32 | 3.807 | 6.057 | 9.864 | 1.852 | 6.148 | 8.001 |
| 8192 | 128 | bfloat16 | 1.486 | 1.691 | 3.176 | 1.072 | 1.480 | 2.552 |
| 16384 | 16 | float32 | 9.192 | 12.614 | 21.806 | 2.406 | 9.649 | 12.056 |
| 16384 | 16 | bfloat16 | 5.226 | 6.151 | 11.378 | 2.062 | 4.944 | 7.006 |
| 16384 | 32 | float32 | 9.532 | 13.203 | 22.736 | 3.084 | 10.315 | 13.399 |
| 16384 | 32 | bfloat16 | 5.255 | 6.136 | 11.391 | 2.410 | 4.923 | 7.334 |
| 16384 | 64 | float32 | 11.504 | 16.966 | 28.469 | 4.207 | 15.017 | 19.224 |
| 16384 | 64 | bfloat16 | 5.354 | 6.219 | 11.573 | 2.853 | 5.127 | 7.980 |
| 16384 | 128 | float32 | 15.174 | 24.364 | 39.538 | 7.314 | 24.284 | 31.599 |
| 16384 | 128 | bfloat16 | 5.515 | 6.432 | 11.947 | 4.202 | 5.510 | 9.713 |
| 32768 | 16 | float32 | 41.604 | 50.271 | 91.875 | 8.856 | 38.206 | 47.062 |
| 32768 | 16 | bfloat16 | 21.233 | 25.347 | 46.580 | 7.472 | 19.129 | 26.601 |
| 32768 | 32 | float32 | 42.953 | 52.093 | 95.046 | 11.166 | 41.229 | 52.395 |
| 32768 | 32 | bfloat16 | 21.307 | 24.593 | 45.901 | 8.696 | 19.192 | 27.888 |
| 32768 | 64 | float32 | 49.811 | 66.168 | 115.979 | 15.867 | 59.065 | 74.932 |
| 32768 | 64 | bfloat16 | 21.725 | 25.153 | 46.878 | 10.247 | 20.068 | 30.316 |
| 32768 | 128 | float32 | 63.958 | 94.980 | 158.938 | 26.951 | 95.530 | 122.481 |
| 32768 | 128 | bfloat16 | 22.418 | 25.903 | 48.322 | 15.810 | 21.549 | 37.359 |
| 65536 | 16 | float32 | OOM | OOM | OOM | OOM | OOM | OOM |
| 65536 | 16 | bfloat16 | OOM | OOM | OOM | 29.151 | 84.180 | 113.331 |
| 65536 | 32 | float32 | OOM | OOM | OOM | OOM | OOM | OOM |
| 65536 | 32 | bfloat16 | OOM | OOM | OOM | 34.063 | 84.229 | 118.292 |
| 65536 | 64 | float32 | OOM | OOM | OOM | OOM | OOM | OOM |
| 65536 | 64 | bfloat16 | OOM | OOM | OOM | 40.097 | 87.523 | 127.620 |
| 65536 | 128 | float32 | OOM | OOM | OOM | OOM | OOM | OOM |
| 65536 | 128 | bfloat16 | OOM | OOM | OOM | 62.323 | 99.858 | 162.181 |

The Triton FlashAttention-2 forward pass is substantially faster than PyTorch at all sequence

lengths.Triton achieves 15.9ms vs 49.8ms with the gap growing as PyTorch's quadratic attention matrix cost dominates.